

MLOps Assignment 4

Apache Airflow Data Pipeline

Student Name	Pruthviraj Pankaj Mohite
PRN	12412293
Class	TY-AIDS-D

1. Objective

The objective of this assignment is to implement and execute an automated data processing workflow using Apache Airflow. The workflow consists of five sequential tasks: extract, validate, process, report, and notify.

2. Workflow Overview

The DAG executes the following task sequence:

Order	Task	Operator	Purpose
1	extract	PythonOperator	Reads input_data.csv and creates extracted_data.csv.
2	validate	PythonOperator	Checks missing values and invalid age records.
3	process	PythonOperator	Removes duplicates, fills missing numerical values, and cleans text fields.
4	report	PythonOperator	Generates the final workflow report.
5	notify	PythonOperator	Prints a successful workflow completion message.

Dependency chain: extract → validate → process → report → notify

3. Brief Explanation of Workflow Execution

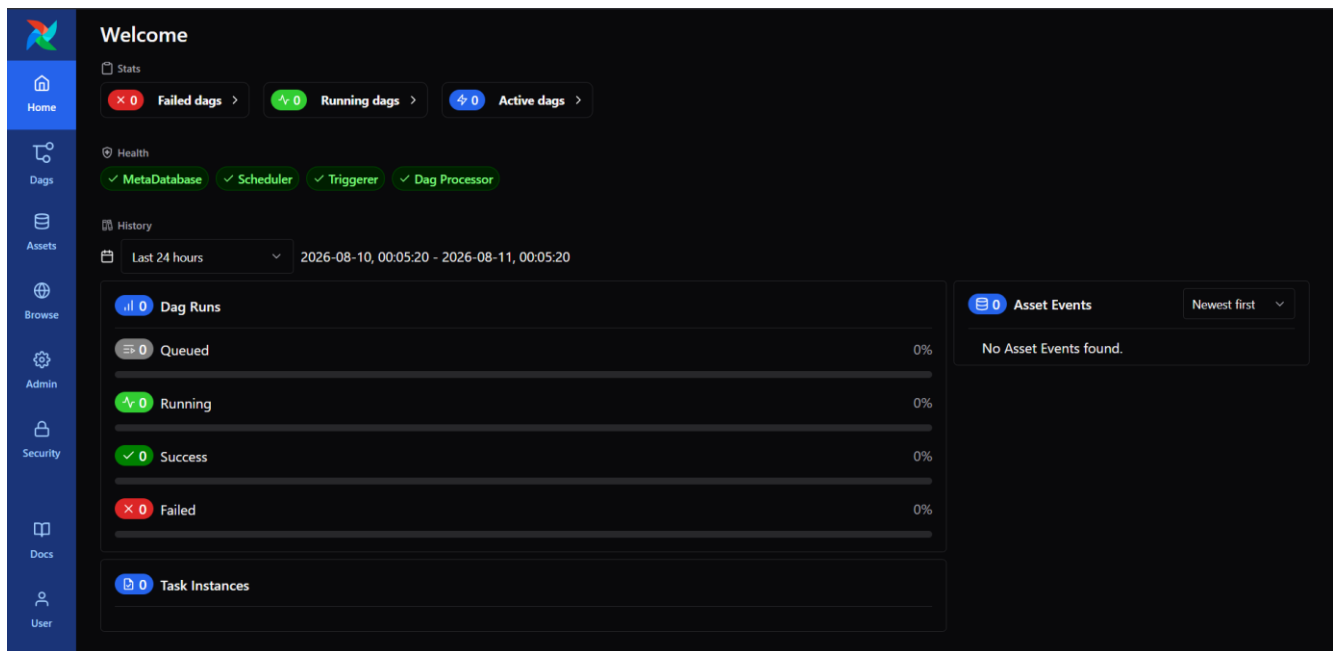
The Airflow DAG `data_pipeline_dag` automates a complete data processing workflow. The `extract` task first reads the input CSV file and writes an extracted copy to the output directory. The `validate` task then examines the extracted dataset for missing values and invalid age records and creates a validation report. Next, the `process` task removes duplicate records, fills missing numerical values using the corresponding median values, and cleans text fields such as names and department names. After processing, the `report` task reads the processed data and validation information to generate a final workflow report. Finally, the `notify` task prints a success message, confirming that the complete workflow has finished successfully.

The successful execution confirmed that all five tasks completed successfully in sequence: `extract`, `validate`, `process`, `report`, and `notify`.

4. Screenshots

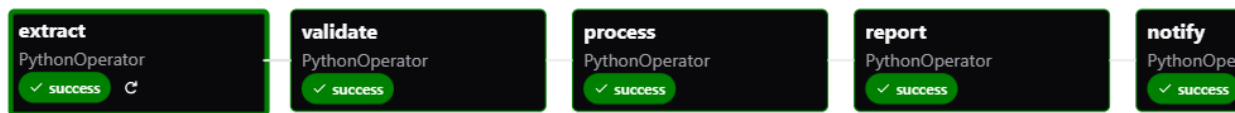
4.1 Airflow Dashboard

A screenshot showing the Airflow interface/dashboard with the `data_pipeline_dag` visible.



4.2 DAG Graph View

A screenshot of the DAG Graph View showing the five tasks and their dependencies.



4.3 Successful Workflow Execution

The screenshot showing the completed DAG run with all five tasks in the SUCCESS state.

The screenshot shows the Apache Airflow DAG Run view for the 'data_pipeline_dag'. The top bar indicates the DAG is 'On' and the run is '2026-08-12, 21:24:35'. The main area displays the DAG graph with all five tasks (extract, validate, process, report, notify) in a 'success' state. The right sidebar shows the run details: Logical Date (2026-08-12, 21:24:35), Run Type (manual), Start (2026-08-12, 21:24:38), End (2026-08-12, 21:33:57), Duration (00:09:18), and Dag Version(s) (v2). Below this, a table lists the tasks and their completion times.

Task ID	Map Index	State	Start
notify		success	2026-08-12 21:33:56
report		success	2026-08-12 21:33:51
process		success	2026-08-12 21:33:33
validate		success	2026-08-12 21:33:11
extract		success	2026-08-12 21:32:58

4.4 Task Logs

 **extract** 2026-08-12, 21:32:58 ✓ success

Operator	Try Number	Start	End
PythonOperator	2	2026-08-12, 21:32:58	2026-08-12, 21:33:11

Duration

00:00:12

DAG Version

v2

Task Tries

1 ✗ 2 ✓

All Log Levels

All Sources

Wrap

Log message source details:

sources=["/opt/airflow/logs/dag_id=data_pipeline_dag/r

2 [2026-08-12, 21:33:02] INFO - DAG bundles loaded: dags-folder: source="airflo

3 [2026-08-12, 21:33:02] INFO - Filling up the DagBag from /opt/airflow/dags/di

4 [2026-08-12, 21:33:11] INFO - Data extracted successfully.: chan="stdout": s

5 [2026-08-12, 21:33:11] INFO - Done. Returned value was: None: source="airflow

6 [2026-08-12, 21:33:11] INFO - Number of records: 8: chan="stdout": source="ti

 **validate** 2026-08-12, 21:33:11 ✓ success

Operator	Start	End	Duration	DAG Version
PythonOperator	2026-08-12, 21:33:11	2026-08-12, 21:33:31	00:00:19	v2

Logs

Rendered Templates

XCom

Audit Logs

Code

Details

Asset Events

All Log Levels

All Sources

Wrap

Log message source details:

sources=["/opt/airflow/logs/dag_id=data_pipeline_dag/run_id>manual__2

2 [2026-08-12, 21:33:25] INFO - DAG bundles loaded: dags-folder: source="airflow.dag_processir

3 [2026-08-12, 21:33:25] INFO - Filling up the DagBag from /opt/airflow/dags/data_pipeline_dag

4 [2026-08-12, 21:33:31] INFO - Done. Returned value was: None: source="airflow.task.operators

5 [2026-08-12, 21:33:31] INFO - : chan="stdout": source="task"

6 [2026-08-12, 21:33:31] INFO - DATA VALIDATION REPORT: chan="stdout": source="task"

7 [2026-08-12, 21:33:31] INFO - =====: chan="stdout": source="task"

8 [2026-08-12, 21:33:31] INFO - : chan="stdout": source="task"

9 [2026-08-12, 21:33:31] INFO - Total Records: 8: chan="stdout": source="task"

10 [2026-08-12, 21:33:31] INFO - : chan="stdout": source="task"

11 [2026-08-12, 21:33:31] INFO - Missing Values:: chan="stdout": source="task"

12 [2026-08-12, 21:33:31] INFO - id 0: chan="stdout": source="task"

13 [2026-08-12, 21:33:31] INFO - name 0: chan="stdout": source="task"

14 [2026-08-12, 21:33:31] INFO - age 1: chan="stdout": source="task"

15 [2026-08-12, 21:33:31] INFO - salary 1: chan="stdout": source="task"

16 [2026-08-12, 21:33:31] INFO - department 0: chan="stdout": source="task"

17 [2026-08-12, 21:33:31] INFO - : chan="stdout": source="task"

18 [2026-08-12, 21:33:31] INFO - Total Missing Values: 2: chan="stdout": source="task"

19 [2026-08-12, 21:33:31] INFO - : chan="stdout": source="task"

20 [2026-08-12, 21:33:31] INFO - Invalid Records: 0: chan="stdout": source="task"

21 [2026-08-12, 21:33:31] INFO - : chan="stdout": source="task"

22 [2026-08-12, 21:33:31] INFO - Validation Status: Completed: chan="stdout": source="task"

23 [2026-08-12, 21:33:31] INFO - : chan="stdout": source="task"

process 2026-08-12, 21:33:33

✓ success

Add a note

Clear Task Instance

Mark Task Instance as...

Operator	Start	End	Duration	DAG Version
PythonOperator	2026-08-12, 21:33:33	2026-08-12, 21:33:50	00:00:17	v2

Logs

Rendered Templates

XCom

Audit Logs

Code

Details

Asset Events

All Log Levels

All Sources

Wrap

Log message source details: sources=["/opt/airflow/logs/dag_id=data_pipeline_dag/run_id>manual__2026-08-12T15:54:37.187598+00:00/task_id=report"]

[2026-08-12, 21:33:43] INFO - DAG bundles loaded: dags-folder: source="airflow.dag_processing.bundles.manager.DagBundlesManager"

[2026-08-12, 21:33:43] INFO - Filling up the DagBag from /opt/airflow/dags/data_pipeline_dag.py: source="airflow.models.dagbag.DagBag"

[2026-08-12, 21:33:50] INFO - Data processing completed.: chan="stdout": source="task"

[2026-08-12, 21:33:50] INFO - Done. Returned value was: None: source="airflow.task.operators.airflow.providers.standard.operator.GenericOperator"

[2026-08-12, 21:33:50] INFO - Processed records: 8: chan="stdout": source="task"

report 2026-08-12, 21:33:51

✓ success

Add a note

Clear Task Instance

Mark Task Instance as...

Operator	Start	End	Duration	DAG Version
PythonOperator	2026-08-12, 21:33:51	2026-08-12, 21:33:54	2.84s	v2

Logs

Rendered Templates

XCom

Audit Logs

Code

Details

Asset Events

All Log Levels

All Sources

Wrap

Log message source details: sources=["/opt/airflow/logs/dag_id=data_pipeline_dag/run_id>manual__2026-08-12T15:54:37.187598+00:00/task_id=report"]

[2026-08-12, 21:33:52] INFO - DAG bundles loaded: dags-folder: source="airflow.dag_processing.bundles.manager.DagBundlesManager"

[2026-08-12, 21:33:52] INFO - Filling up the DagBag from /opt/airflow/dags/data_pipeline_dag.py: source="airflow.models.dagbag.DagBag"

[2026-08-12, 21:33:54] INFO - Done. Returned value was: None: source="airflow.task.operators.airflow.providers.standard.operator.GenericOperator"

[2026-08-12, 21:33:54] INFO - : chan="stdout": source="task"

[2026-08-12, 21:33:54] INFO - AIRFLOW DATA PIPELINE REPORT: chan="stdout": source="task"

[2026-08-12, 21:33:54] INFO - =====: chan="stdout": source="task"

[2026-08-12, 21:33:54] INFO - : chan="stdout": source="task"

[2026-08-12, 21:33:54] INFO - Workflow: data_pipeline_dag: chan="stdout": source="task"

[2026-08-12, 21:33:54] INFO - : chan="stdout": source="task"

[2026-08-12, 21:33:54] INFO - Number of Records Processed:: chan="stdout": source="task"

[2026-08-12, 21:33:54] INFO - 8: chan="stdout": source="task"

[2026-08-12, 21:33:54] INFO - : chan="stdout": source="task"

[2026-08-12, 21:33:54] INFO - Missing Values Identified:: chan="stdout": source="task"

[2026-08-12, 21:33:54] INFO - : chan="stdout": source="task"

[2026-08-12, 21:33:54] INFO - DATA VALIDATION REPORT: chan="stdout": source="task"

[2026-08-12, 21:33:54] INFO - =====: chan="stdout": source="task"

[2026-08-12, 21:33:54] INFO - : chan="stdout": source="task"

[2026-08-12, 21:33:54] INFO - Total Records: 8: chan="stdout": source="task"

[2026-08-12, 21:33:54] INFO - : chan="stdout": source="task"

[2026-08-12, 21:33:54] INFO - Missing Values:: chan="stdout": source="task"

[2026-08-12, 21:33:54] INFO - id 0: chan="stdout": source="task"

[2026-08-12, 21:33:54] INFO - name 0: chan="stdout": source="task"

MLOps Assignment 4 — Apache Airflow Data Pipeline

 **notify** 2026-08-12, 21:33:56 ✓ success   

Operator	Start	End	Duration	DAG Version
PythonOperator	2026-08-12, 21:33:56	2026-08-12, 21:33:56	0.59s	v2

Logs
Rendered Templates
XCom
Audit Logs
Code
Details
Asset Events

All Log Levels 

All Sources 

Wrap



► Log message source details: sources=["/opt/airflow/logs/dag_id=data_pipeline_dag/run_id>manual_

2 [2026-08-12, 21:33:56] INFO - DAG bundles loaded: dags-folder: source="airflow.dag_processi

3 [2026-08-12, 21:33:56] INFO - Filling up the DagBag from /opt/airflow/dags/data_pipeline_da

4 [2026-08-12, 21:33:56] INFO - SUCCESS: Data pipeline workflow completed successfully!: char

5 [2026-08-12, 21:33:56] INFO - Done. Returned value was: None: source="airflow.task.operator

6. Airflow DAG Python File

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime, timedelta
import os

BASE_DIR = "/opt/airflow"

DATA_FILE = os.path.join(
    BASE_DIR,
    "data",
    "input_data.csv"
)

EXTRACTED_FILE = os.path.join(
    BASE_DIR,
    "output",
    "extracted_data.csv"
)

PROCESSED_FILE = os.path.join(
    BASE_DIR,
    "output",
    "processed_data.csv"
)

VALIDATION_FILE = os.path.join(
    BASE_DIR,
```

```

        "output",
        "validation_report.txt"
    )

REPORT_FILE = os.path.join(
    BASE_DIR,
    "output",
    "final_report.txt"
)

def extract():
    import pandas as pd

    os.makedirs(
        os.path.join(BASE_DIR, "output"),
        exist_ok=True
    )

    df = pd.read_csv(DATA_FILE)

    df.to_csv(
        EXTRACTED_FILE,
        index=False
    )

    print("Data extracted successfully.")
    print(f"Number of records: {len(df)}")

def validate():
    import pandas as pd

    df = pd.read_csv(EXTRACTED_FILE)

    missing_values = df.isnull().sum()
    total_missing = missing_values.sum()

    invalid_records = 0

    if "age" in df.columns:
        invalid_records += (
            (df["age"].notna()) &
            (df["age"] <= 0)
        ).sum()

    validation_text = f"""
DATA VALIDATION REPORT

```

```

=====

Total Records: {len(df)}

Missing Values:
{missing_values.to_string()}

Total Missing Values: {total_missing}

Invalid Records: {invalid_records}

Validation Status: Completed
"""

    with open(VALIDATION_FILE, "w") as file:
        file.write(validation_text)

    print(validation_text)

def process():
    import pandas as pd

    df = pd.read_csv(EXTRACTED_FILE)

    # Remove duplicate records
    df = df.drop_duplicates()

    # Fill missing numerical values
    if "age" in df.columns:
        df["age"] = df["age"].fillna(
            df["age"].median()
        )

    if "salary" in df.columns:
        df["salary"] = df["salary"].fillna(
            df["salary"].median()
        )

    # Clean text fields
    if "name" in df.columns:
        df["name"] = df["name"].str.strip()

    if "department" in df.columns:
        df["department"] = (
            df["department"]
            .str.strip()
            .str.title()

```



```

    )

    df.to_csv(
        PROCESSED_FILE,
        index=False
    )

    print("Data processing completed.")
    print(f"Processed records: {len(df)}")

def report():
    import pandas as pd

    df = pd.read_csv(PROCESSED_FILE)

    validation = ""

    if os.path.exists(VALIDATION_FILE):
        with open(VALIDATION_FILE, "r") as file:
            validation = file.read()

    report_text = f"""
AIRFLOW DATA PIPELINE REPORT
=====

Workflow: data_pipeline_dag

Number of Records Processed:
{len(df)}

Missing Values Identified:
{validation}

Processing Completion Status:
SUCCESS

Report Generation Status:
COMPLETED
"""

    with open(REPORT_FILE, "w") as file:
        file.write(report_text)

    print(report_text)

def notify():
    print(

```

```

        "SUCCESS: Data pipeline workflow completed successfully!"
    )

default_args = {
    "owner": "airflow",
    "depends_on_past": False,
    "retries": 1,
    "retry_delay": timedelta(minutes=5),
    "execution_timeout": timedelta(minutes=30),
}

with DAG(
    dag_id="data_pipeline_dag",
    default_args=default_args,
    start_date=datetime(2026, 1, 1),
    schedule="@daily",
    catchup=False,
    dagrun_timeout=timedelta(hours=1),
    tags=["MLOps", "Assignment4"]
) as dag:

    t1 = PythonOperator(
        task_id="extract",
        python_callable=extract
    )

    t2 = PythonOperator(
        task_id="validate",
        python_callable=validate
    )

    t3 = PythonOperator(
        task_id="process",
        python_callable=process
    )

    t4 = PythonOperator(
        task_id="report",
        python_callable=report
    )

    t5 = PythonOperator(
        task_id="notify",
        python_callable=notify
    )

    t1 >> t2 >> t3 >> t4 >> t5

```